

CS443 Project 2: How This Project Works

Big Picture

This project builds image-classification neural networks in low-level TensorFlow. Its main goal is to test predictive coding networks (PCNs), which are inspired by the idea that a brain-like system learns by trying to predict what it should see next and then correcting its mistakes.

What Problem It Solves

The code teaches a model to look at an image and decide which class it belongs to, such as a digit in MNIST or an object in CIFAR-10.

Main Flow

1. Load data: `image_datasets.py` downloads MNIST or CIFAR-10, scales pixel values, and can flatten images when needed.
2. Build a network: `dense_pcn.py` creates a fully connected predictive coding network, `conv_pcn.py` creates an image-based version, and `network.py` provides shared training and evaluation logic.
3. Run the network: in a standard network, data mostly moves forward layer by layer. In a predictive coding network, each layer or block also tries to reconstruct what came before it.
4. Measure error: classification models use cross-entropy loss. Dense PCNs can also use predictive loss, which measures how well layers predict each other.
5. Update weights: TensorFlow computes gradients and the Adam optimizer adjusts the weights so the model improves over time.
6. Evaluate results: the code tracks loss and accuracy on validation or test data.

What Makes This Project Different

The key idea is predictive coding:

- A layer makes a guess about the layer below it.
- The model calculates the prediction error.
- That error is used to update the layer's internal state for several small steps.

In the convolutional version, this happens inside `ConvPCNBlock`:

- A `Conv2D` layer builds features.
- A `Conv2DTranspose` layer tries to reconstruct the previous input.
- The block repeats this correction process before passing its final state forward.

Key Files

- `network.py`: shared training loop, loss, accuracy, evaluation
- `image_datasets.py`: dataset loading and preprocessing
- `dense_pcn.py` and `dense_pcn_layer.py`: dense predictive coding model
- `conv_pcn.py` and `conv_pcn_block.py`: image-based predictive coding model
- `layers.py` and `conv_layers.py`: reusable dense and convolutional layers
- `*.ipynb`: experiments, analysis, and model runs

Simple Mental Model

The project looks at an image, makes a prediction, checks how wrong it was, corrects itself, and repeats. Predictive coding adds one more idea: each part of the network also checks whether its own expectations match what it received.